Amortizirana časovna zahtevnost

Operacija ima amort. čas zahtevnost c, če kakršno koli zaporedje n operacij zahteva največjemu $c \cdot n$ enot časa

Primer: Dvojiški števec predstavljen s tabelo ničel in enic B, ki predstavlja število $\sum B[i] \cdot 2^i$

Za posamezno povečanje $O(\log n)$ časa za n korakov potrebujemo $O(n \log(n))$ za en korak je amort. čas. zahtevnost $O(\log(n))$

Agregacijska metoda

$$C = \frac{\sum_{i=n}^{n} c_i}{n}$$

1. Dvojiški števec

$$\sum c_i = n + \lfloor \tfrac{n}{2} \rfloor + \ldots + \lfloor \tfrac{n}{2^{\log n}} \rfloor = \leq \sum_{k=0}^{\infty} \frac{n}{2^k} = 2n$$

$\Rightarrow$ $C = 2$ Amortizirana čas. zah. je $O(1)$

2 Dinamična tabela

Pri vsakem koraku vstavimo 1 element

Na koraku $2^k + 1$ prestavimo te 2^k el.

$$\sum c_i = n + 1 + 2 + 4 + 8 + \ldots + 2^{\lfloor \log n \rfloor} \leq$$

$$\leq n + \sum_{i=0}^{\lfloor \log n \rfloor} 2^i = n + 2^{\lfloor \log n \rfloor + 1} - 1$$

$$\leq n + 2^{\log(n)+1} = 3n$$

Računska metoda

Ideja: Operacije lahke „plačajo" del stroška kasnejših operacij

1. Dvojiški števec
   - Vsaka sprememba iz 0 na 1 plača 2 enoti. Eno zase, eno za morebitno spremembo 1→0
   - Sprememba 1→0 pokriti privarčevana sredstva

   Operacija bo plačala $\hat{c}_i$ tako, da bo $\sum c_i \leq \sum \hat{c}_i$

Na vsakem koraku plačamo natanko 2 enoti.

Hkrati je ta dovolj, saj vsaka sprememba 1→0 sled. spremembi 0→1

3. Unija z rangom

   Glej PSA1

   Dokažimo da je amortizirana čas. zah. $O(\log(n))$

$I_0 = [1, 1]$ $I_2 = [2, 2]$ $I_3 = [3, 4]$ $I_3 = 5, 16]$

$$I_k = [2^l + 1, 2^{2^l}] = \{x \in \mathbb{N} \mid \log^* x = k\}$$

$\forall$ 26.2 manyhola

5.3

Predvorba statičnih podatkovnih struktur v dinamične

dinamična .... dodajamo in vzemamo elemente
poldinamična .... samo eno ali drugo
statična .... samo gledamo elemente

Elemente lahko shranimo v statično tabelo.
Ko odstranimo element, označimo v neki drugi tabeli vrednost False, da ga ne moremo več vrniti

Recimo, da imamo P.S. nad elementi iz množice X s stopnjo $S(X)$, ki podpira
- predpripravo $\iota : P(X) \to S(X)$
- poizvedbe $\varphi : S(X) \times Q \to A$

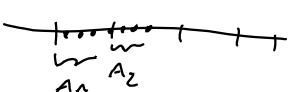
tako da poizvedbe lahko razpadejo preko operacije $\oplus : \mathcal{R} \times \mathcal{R} \to \mathcal{R}$

da je

$$\varphi(\bigcup(A_1 \cup A_2), g) = \varphi(\iota(A_1), g) \oplus \varphi(\iota(A_2), g)$$

Primeri:
- iskanje elementa v tabeli
  - ι .... uredi tabelo
  - φ .... poišče z bisekcijo
  - $\oplus$ .... $\vee$
- iskanje najbližje točke v ravnini v $A^{kom} \subseteq \mathbb{R}^k$
  - ι .... k-d drevo
  - φ ..... najbližja točka
  - $\oplus$ ..... izbere izmed dveh točk bližnjo

Protiprimer
Število enotnih intervalov s krajišči v A



ne razpade

Ideja je, da vodimo svoje $L_0, L_1, \dots, L_k$

v L_i je 2^i elementov, ki so urejeni.

Nov element dodajamo na najnižji nivo

če je zasedeno, jih potisnemo skupaj in demo gor

Naj bo • časovna zahtevnost $L(A)$ enaka $P(n)$ $n = |A|$

• čas. zaht. $p(L(A), g)$ je $Q(n)$

• Naj bo P superaditivna $(P(m+n) \geq P(m) + P(n))$

Tedaj je amortizirana čas. zaht. vstavljanja enaka

$$I'(n) = \begin{cases} O(P(n)) & P(n) \in \Theta(n^{1+\varepsilon}) \quad \varepsilon > 0 \\ P(n)\, O(\log n); & \text{v vseh primerih} \end{cases}$$

' označuje zahtevnosti na novi strukturi

Časovna zahtevnost poizvedb:

$$Q'(n) = \begin{cases} O(Q(n)) & ; \quad Q(n) \in \Theta(n^{\varepsilon}) \quad \varepsilon > 0 \\ Q(n)\, O(\log n) & ; \quad \text{v vseh primerih} \end{cases}$$